

# Cut-and-Choose Issuance for Compact Lattice-Based Keyed-Verification Credentials

Anonymous draft

Working draft — security claims are provisional

## Abstract

We investigate a cut-and-choose variant of the keyed-verification blind signature of Beullens, Lyubashevsky, Nguyen, and Seiler (BLNS23). The goal is to remove the expensive first-round zero-knowledge proof that a user-supplied lattice commitment contains correctly evaluated hash functions. A user sends two commitments in each of  $\kappa$  independent lanes. After an unpredictable challenge, the user opens one commitment per lane and the signer evaluates only the complementary commitments. Crucially, the signer releases a single aggregate response rather than the lane responses. A direct chosen-input attack must then evade all  $\kappa$  checks to remove every honestly formed mask, which occurs with probability  $2^{-\kappa}$ . The resulting credential authenticates a vector of hidden messages and is verified with the tuple of lane keys. We give the construction, its correctness calculation, and a conditional security argument. The central unresolved question is a robust “one honest lane” lemma for adversarially correlated lattice inputs; until that lemma is proved under a standard assumption, the construction should be regarded as a candidate rather than a secure scheme.

## 1 Introduction

BLNS23 gives a compact keyed-verification blind signature whose issuance protocol contains two non-interactive zero-knowledge proofs. The user’s proof  $\pi_1$  establishes knowledge of short  $\mathbf{r}, \mathbf{e}_2$  and a message  $m$  satisfying

$$\rho = H_\rho(\mathbf{r}, \mathbf{e}_2), \quad \mathbf{c} = B\mathbf{r} + \mathbf{e}_2 + H_R(m, \rho).$$

This proof prevents a malicious user from choosing  $\mathbf{c}$  so as to query the signing secret on a scaled unit vector. Its statement includes hash evaluation and is the principal issuance bottleneck.

We ask whether statistical cut-and-choose can replace  $\pi_1$ . A naive construction with  $N$  commitments, of which  $N - 1$  are opened, has soundness error  $1/N$  and therefore requires an infeasible  $N \geq 2^\lambda$ . Our candidate instead uses  $\kappa$  pairs and returns only one aggregate response. If a single honestly generated unopened lane computationally masks all malformed lanes, a useful chosen-input attack must corrupt every unopened lane simultaneously. Parallel repetition then gives error  $2^{-\kappa}$ . Applying Fiat–Shamir to the full vector of pair commitments places the challenge and openings in the user’s first message. The signer returns one aggregate response, giving a two-message, round-optimal issuance protocol.

The construction changes the message space. A credential authenticates a vector  $(m_0, \dots, m_{\kappa-1})$ , not one message copied across all lanes. The opened message in every pair is a decoy; the complementary message remains hidden during issuance and is disclosed at presentation. An application may derive a single payload by hashing the ordered vector.

**Status.** Correctness and the elementary cut-and-choose probability are immediate. A complete proof of blindness and one-more unforgeability is not supplied here. In particular, Section 7 identifies the one-honest-lane lemma, related-input security, selective failure, and concurrency as open obligations.

## 2 Preliminaries and notation

Let  $R_q = \mathbb{Z}_q[X]/(X^d + 1)$ . Vectors are written in bold and all ring operations are modulo  $q$ . Let  $D_s, D_r, D_e$  be bounded short distributions, and let  $\lceil \cdot \rceil_p$  retain the prescribed high-order coefficient bits. The security parameter and number of cut-and-choose lanes are denoted  $\lambda$  and  $\kappa$ . Classically,  $\kappa = 128$  targets 128-bit challenge soundness; in the quantum random-oracle model,  $\kappa = 256$  is the conservative starting point because of generic quadratic search speedups.

For lane  $i$ , the signer samples

$$\mathbf{s}_i \leftarrow \$ D_s, \quad \mathbf{e}_{1,i} \leftarrow \$ D_e,$$

publishes a random matrix  $B_i$  and

$$\mathbf{t}_i^T = \mathbf{s}_i^T B_i + \mathbf{e}_{1,i}^T.$$

The keyed verification key is the tuple  $\mathbf{sk} = (\mathbf{s}_0, \dots, \mathbf{s}_{\kappa-1})$ . It is generally *not* sufficient to retain only  $\sum_i \mathbf{s}_i$ , because different lanes authenticate different hash inputs.

We require a batched lattice NIZK  $\Pi_{\text{resp}}$  and a 384-bit collision-resistant hash  $H_{\text{com}}$ . The latter gives approximately 128-bit collision resistance against generic quantum collision search. The NIZK contains no hash preimage relation.

### 3 Construction

#### 3.1 Commit phase

The user independently constructs two branches  $b \in \{0, 1\}$  in each lane  $i \in \{0, \dots, \kappa - 1\}$ . It samples a 32-byte seed  $z_{i,b}$  and uses a domain-separated XOF and canonical deterministic sampler to derive

$$(m_{i,b}, \mathbf{r}_{i,b}, \mathbf{e}_{2,i,b}) = G(\text{sid}, i, b, z_{i,b}),$$

where  $m_{i,b}$  is 32 bytes and the two vectors have the prescribed bounded distributions. It computes

$$\begin{aligned} \rho_{i,b} &= H_\rho(\text{sid}, i, b, \mathbf{r}_{i,b}, \mathbf{e}_{2,i,b}), \\ \mathbf{u}_{i,b} &= H_R(\text{credential-lane}, i, m_{i,b}, \rho_{i,b}), \\ \mathbf{c}_{i,b} &= B_i \mathbf{r}_{i,b} + \mathbf{e}_{2,i,b} + \mathbf{u}_{i,b}. \end{aligned}$$

The session identifier  $\text{sid}$  includes the protocol version, issuer, key epoch, and fresh contributions from both parties. The user commits once to the complete canonically ordered vector of branches:

$$C = H_{\text{com}}(\text{protocol}, \text{issuer}, \text{pk}, \text{sid}, \text{Encode}(\mathbf{c}_{0,0}, \mathbf{c}_{0,1}, \dots, \mathbf{c}_{\kappa-1,0}, \mathbf{c}_{\kappa-1,1})).$$

The encoding includes the branch count, dimensions, and unambiguous lengths. Thus one 48-byte digest binds all  $2\kappa$  branches and their ordering before challenge selection.

#### 3.2 Fiat–Shamir challenge and opening

After all commitments are fixed, the user derives

$$\chi = (\chi_0, \dots, \chi_{\kappa-1}) = H_{\text{chal}}(\text{protocol}, \text{issuer}, \text{pk}, \text{sid}, C) \in \{0, 1\}^\kappa.$$

For every lane, the user opens branch  $\chi_i$  by disclosing only the seed  $z_{i,\chi_i}$ , and sends the full vector  $\mathbf{c}_{i,1-\chi_i}$  for the complementary branch. On receipt, the signer first recomputes  $\chi$  from the transmitted  $C$  and verifies that each seed and full vector has the branch role prescribed by  $\chi_i$ . It expands every disclosed seed, recomputes the opened  $\mathbf{c}_{i,\chi_i}$ , and combines these with the received complementary vectors in canonical order. It recomputes the global commitment  $C$  over all 512 reconstructed branches. It also checks both hash computations, the linear relation, all norm bounds, canonical encodings, and domain separators for every opened branch. It aborts without producing any secret-dependent output if any check fails. Define the selected branch  $b_i^* = 1 - \chi_i$  and abbreviate  $\mathbf{c}_i^* = \mathbf{c}_{i,b_i^*}$ .

The selected seeds are retained privately by the user. At presentation the user reveals only  $(m_i^*, \rho_i^*)$ , never  $z_{i,b_i^*}$ : revealing a selected seed would let the issuer reconstruct  $\mathbf{r}_i^*, \mathbf{e}_{2,i}^*$  and link the presented credential to its issuance commitment exactly. The XOF, rejection-sampling order, byte encoding, and maximum iteration policy are part of the protocol specification so that seed expansion is deterministic and cross-platform reproducible.

Thus issuance has the two-message form

$$\text{User} \longrightarrow \text{Signer} : C, \{z_{i,\chi_i}, \mathbf{c}_{i,1-\chi_i}\}_i, \quad \text{Signer} \longrightarrow \text{User} : h, \pi_{\text{resp}}.$$

Fiat–Shamir removes the challenge round but permits offline transcript grinding. Prescribing  $k$  challenge bits costs about  $2^k$  classical random-oracle queries and about  $2^{k/2}$  generic quantum queries. A tight QROM proof is therefore required for the post-quantum parameter choice. A fresh signer nonce limits precomputation, but adds a message unless it is already supplied by an authenticated session setup.

### 3.3 Aggregate response

The signer derives a bounded aggregate error deterministically from the entire selected transcript,

$$e = H_{D_e}(K, \text{sid}, \mathbf{c}_0^*, \dots, \mathbf{c}_{\kappa-1}^*),$$

and computes only

$$h = \sum_{i=0}^{\kappa-1} \mathbf{s}_i^T \mathbf{c}_i^* + e.$$

No individual lane value is released or made observable through timing or errors. The signer proves the batched relation

$$\mathcal{R}_{\text{resp}} : \begin{cases} h = \sum_i \mathbf{s}_i^T \mathbf{c}_i^* + e, \\ \mathbf{t}_i^T = \mathbf{s}_i^T B_i + \mathbf{e}_{1,i}^T, \\ \|\mathbf{s}_i\| \leq \beta_s, \|\mathbf{e}_{1,i}\| \leq \beta_e, \|e\| \leq \beta_e. \end{cases} \quad \text{for every } i,$$

The witness is the tuple of all component secrets and errors. Proving knowledge only of  $S = \sum_i \mathbf{s}_i$  is insufficient because the  $\mathbf{c}_i^*$  differ. The important efficiency observation is that  $\mathcal{R}_{\text{resp}}$  consists only of lattice-linear relations and norm bounds; it contains no hash evaluation.

### 3.4 Unblinding and presentation

After verifying the response proof, the user computes

$$v = \left\lceil h - \sum_i \mathbf{t}_i^T \mathbf{r}_i^* \right\rceil_p$$

and stores

$$\sigma = ((m_i^*, \rho_i^*)_{i=0}^{\kappa-1}, v), \quad (m_i^*, \rho_i^*) = (m_{i,b_i^*}, \rho_{i,b_i^*}).$$

At presentation, the keyed verifier accepts iff

$$v = \left\lceil \sum_i \mathbf{s}_i^T H_R(\text{credential-lane}, i, m_i^*, \rho_i^*) \right\rceil_p.$$

The issuance session identifier must not enter the presented lane hash: doing so would give the issuer a trivial linkage handle. Session binding belongs in the cut-and-choose commitments and challenge, while issuance and presentation use the same credential-domain lane hash. An application-level payload can be defined as

$$M = H_{\text{final}}(\text{issuer}, \text{type}, (m_i^*, \rho_i^*)_{i=0}^{\kappa-1}).$$

## 4 Correctness

For an honestly generated selected branch,

$$\mathbf{c}_i^* = B_i \mathbf{r}_i^* + \mathbf{e}_{2,i}^* + \mathbf{u}_i^*.$$

Consequently,

$$h - \sum_i \mathbf{t}_i^T \mathbf{r}_i^* = \sum_i \mathbf{s}_i^T \mathbf{u}_i^* + \sum_i (\mathbf{s}_i^T \mathbf{e}_{2,i}^* - \mathbf{e}_{1,i}^T \mathbf{r}_i^*) + e.$$

Thus issuance and verification agree whenever the aggregate error does not cross a rounding boundary. Compared with BLNS23, the error contains a sum of  $\kappa$  lane errors. Parameters therefore cannot be copied unchanged: the noise bound grows at least as  $O(\sqrt{\kappa})$  for independent errors and as  $O(\kappa)$  under a worst-case norm bound. This may require a larger  $q$ , a smaller rounding modulus  $p$ , or narrower lane distributions.

## 5 Cut-and-choose analysis

Call a branch *openable* if its disclosed witness passes every opening check. An accepted transcript has at least one openable branch in every pair. Suppose the adversary prepares one openable and one malformed branch in each of  $k$  attacked lanes. All malformed branches are selected exactly when the challenge opens the openable branch in every attacked lane, hence

$$\Pr[\text{all attacked selections malformed}] = 2^{-k}.$$

In particular, removing every honest mask across  $\kappa$  lanes has probability  $2^{-\kappa}$  per attempt, and at most  $Q2^{-\kappa}$  over  $Q$  attempts by a union bound.

For the direct scaled-unit attack, the adversary sets every malformed selected commitment to  $\alpha \mathbf{e}_j$ . If all lanes select those branches, then

$$h = \alpha \sum_i s_{i,j} + e,$$

so rounding  $h/\alpha$  exposes a coordinate of the summed key. The event has probability  $2^{-\kappa}$  per independent classical trial, while generic quantum Fiat–Shamir grinding costs about  $2^{\kappa/2}$ . If any unopened lane is honest, its term remains in the aggregate and is intended to computationally mask the malformed terms.

This last sentence is an assumption to be proved, not a consequence of the elementary probability calculation.

### 5.1 A partial masking lemma

The desired statement can be separated into an information-selection claim and a lattice-pseudorandomness claim. Let  $\text{Valid}(i, b)$  denote the existence of an opening satisfying all lane relations. For a fixed vector of branch commitments, there is at most one challenge  $\chi$  for which every opened branch is valid while every selected branch is invalid: in every pair it must open the unique valid branch. Consequently, a classical adversary making  $q_H$  Fiat–Shamir trials reaches this all-invalid event with probability at most

$$q_H 2^{-\kappa},$$

up to collisions and challenge bias. In the QROM the corresponding generic bound has the expected quadratic loss,  $O(q_H^2 2^{-\kappa})$ .

Now fix a selected lane  $j$  for which a valid witness  $(m_j, \rho_j, \mathbf{r}_j, \mathbf{e}_{2,j})$  is available to the reduction. Give the distinguisher all secrets outside lane  $j$ ; this only strengthens it. It can subtract their contributions from the aggregate response. What remains is the BLNS23 lane response

$$\mathbf{s}_j^T (B_j \mathbf{r}_j + \mathbf{e}_{2,j} + H_R(m_j, \rho_j)) + e.$$

Using  $\mathbf{t}_j^T \mathbf{r}_j$  to account for the blinding term and programming the keyed error exactly as in the BLNS23 HMLWE hybrid leaves a hashed MLWE evaluation under  $\mathbf{s}_j$ . It is indistinguishable from a uniform ring element under HMLWE. Since adding known or independently keyed malformed-lane contributions preserves uniformity, the entire aggregate is pseudorandom.

**Lemma 1** (Extracted one-honest-lane masking, informal). *In the random-oracle model, suppose a simulator obtains the witness of one valid selected lane in every accepted issuance, the lane keys are independent, and the aggregate error can be programmed as required by the BLNS23 HMLWE hybrid. Then replacing the aggregate response of that issuance by a uniform ring element is computationally indistinguishable under HMLWE, with a polynomial loss for guessing the lane and signing query.*

The classical extraction step has a direct two-fork argument. Fix the user’s state immediately before its Fiat–Shamir query, and obtain two accepting transcripts with the same branch commitments but distinct challenges  $\chi \neq \chi'$ . There is an index  $j$  such that  $\chi_j \neq \chi'_j$ . Because the challenges are binary,

$$\chi'_j = 1 - \chi_j = b_j^*.$$

The second transcript therefore opens exactly the branch that was selected and left unopened in the first transcript. Verification of that opening yields  $(m_j^*, \rho_j^*, \mathbf{r}_j^*, \mathbf{e}_{2,j}^*)$ , the witness required by the HMLWE reduction.

**Lemma 2** (Two-fork selected-lane extraction). *Assuming binding of the global branch commitment, two accepting transcripts with identical pair commitments and distinct challenge vectors reveal a valid witness for at least one branch selected in the first transcript (and symmetrically for one branch selected in the second).*

The proof is the coordinate argument above and is information theoretic. If a fixed commitment vector admits only one accepting challenge, its Fiat–Shamir success probability is  $2^{-\kappa}$ . If a classical polynomial-time prover succeeds with non-negligible probability, the standard forking strategy—rewind at the challenge query and answer it independently—obtains two distinct accepting challenges with non-negligible probability, subject to the usual forking loss in the number of random-oracle queries. Combining this extractor with the preceding HMLWE hybrid gives a plausible classical-ROM proof route.

What remains unresolved is making the extractor online and adaptive for polynomially many signing queries and concurrent sessions, and then lifting it to the QROM. Generic Fiat–Shamir proofs of knowledge do not automatically provide adaptive multi-theorem extraction, and quantum rewinding cannot simply copy the prover state. A QROM proof would instead need a measure-and-reprogram argument and a suitable adaptive special-soundness theorem [4].

One clean but expensive proof device is to add an extractable commitment to each branch witness, or attach an encryption-to-the-sky of each branch witness. The security reduction then knows the extraction key and can recover the selected valid witness, while no such key exists in the real protocol. This avoids adaptive rewinding but adds  $2\kappa$  large lattice ciphertexts and may erase the proposed efficiency advantage. Finding a lightweight extractability mechanism is therefore part of the contribution.

## 5.2 Counterexamples to nearby variants

The analysis already gives concrete counterexamples to several tempting simplifications:

1. If individual lane responses are released, attacking one pair succeeds with probability  $1/2$  and leakage accumulates across sessions.
2. If all lanes reuse the same secret  $\mathbf{s}$ , a malformed lane can be correlated with an honest lane so that their coefficients cancel. Independent lane keys are essential.
3. If the Fiat–Shamir hash omits any branch commitment, public key, lane index, or protocol domain, the omitted value can be chosen after the challenge.
4. If  $\kappa = 128$  is used against quantum users, Grover-style transcript grinding gives only approximately 64 bits of generic work.
5. If responses or partial sums are emitted before all openings pass, one malformed lane again becomes a probability- $1/2$  key-leakage oracle.

**Conjecture 1** (One-honest-lane aggregate security). *Let the adversary choose all branch pairs adaptively before receiving a uniform challenge, subject only to successful openings. Conditioned on at least one selected branch being honestly distributed, aggregate responses from*

$$h = \sum_i \mathbf{s}_i^T \mathbf{c}_i^* + e$$

*do not enable recovery of the key tuple, its useful aggregate, or a one-more credential forgery, even across polynomially many concurrent sessions with adversarially correlated fresh inputs.*

A paper claiming security must replace this conjecture with a reduction to a standard MLWE/MSIS-style problem, or state and justify a precise new assumption.

## 6 Blindness intuition

The issuer learns only the opened decoy branch in each lane. The credential contains the complementary messages. At presentation the issuer learns  $(m_i^*, \rho_i^*)_i$  and can subtract their hashes from the stored unopened commitments, but deciding whether

$$\mathbf{c}_i^* - H_R(m_i^*, \rho_i^*) = B_i \mathbf{r}_i^* + \mathbf{e}_{2,i}^*$$

for short unknown values should be hidden by the underlying MLWE distribution. This suggests issuance–presentation unlinkability when opened and unopened messages have indistinguishable distributions.

The response proof is necessary against a malicious issuer. Without it, the issuer may use malformed component keys or signer-chosen errors to watermark credentials. The proof must establish key-parameter consistency for every lane and at least constrain the aggregate error norm. Proving deterministic PRF generation of the error would add a nonlinear relation and may undermine the claimed efficiency; alternatively, one must prove that every permitted bounded error is erased by rounding except with negligible probability and cannot be used for selective-failure tagging.

## 7 Open security obligations

1. **One honest lane.** Prove the conjecture above for adversarially correlated, adaptively selected commitments. Complete the classical adaptive multi-session forking reduction, then obtain a QROM extractor without assuming an impractically large extractable commitment.
2. **Related inputs across sessions.** Rule out differential attacks using fresh but algebraically related commitment vectors. Exact replay prevention alone is insufficient.
3. **Selective failure.** Show that malicious challenges, aborts, malformed response proofs, and rounding failures do not identify the hidden branch vector at later presentation.
4. **Concurrency and reset security.** Bind every message to a unique session and analyze interleaved, replayed, reset, and rolled-back sessions.
5. **One-more unforgeability.** Define the vector-message game and handle linear combinations or splicing of components from multiple issued credentials.
6. **Malicious-key security.** Prove that the batched response NIZK gives key-parameter consistency and prevents subliminal tagging.
7. **Noise growth.** Instantiate parameters for the aggregate error and bound the rounding-failure probability at the target security level.
8. **Fiat–Shamir and challenge entropy.** Give ROM and QROM analyses accounting for adaptive transcript grinding, multi-target attacks, malicious abort-and-restart strategies, and concurrency. Determine whether  $\kappa = 256$  tightly achieves 128-bit post-quantum soundness.
9. **Side channels and atomicity.** Ensure that no individual lane response, partial sum, failing index, or timing signal is released.

## 8 Preliminary efficiency discussion

The candidate removes the hash-heavy user NIZK, replacing it with  $2\kappa$  hash commitments and  $\kappa$  seeded openings. For a classical target with  $\kappa = 128$ , issuance sends 256 lattice commitments and opens 128 witnesses; the conservative post-quantum starting point doubles those figures. Fiat–Shamir preserves two-message issuance despite this bandwidth. The signer performs  $\kappa$  hash-and-relation checks and one aggregate evaluation. The response NIZK is larger than the single-lane BLNS23  $\pi_2$ , but is batchable and purely lattice algebraic.

### 8.1 Concrete communication estimate

We instantiate the wire calculation with the BLNS23 keyed-verification values  $d = 64$ ,  $n = 93$ ,  $\lceil \log_2 q \rceil = 152$ ,  $p = 4$ , and 32-byte messages and tags. We take  $\kappa = 256$  as the conservative starting point for 128-bit post-quantum Fiat–Shamir grinding resistance. Ring elements are packed at 152 bits per coefficient. Branch openings are 32-byte seeds and the single global branch commitment is a 48-byte digest.

One commitment vector has exact packed length

$$|c| = nd \lceil \log_2 q \rceil = 93 \cdot 64 \cdot 152 = 904,704 \text{ bits} = 113,088 \text{ bytes}.$$

Only the  $\kappa = 256$  unopened vectors are transmitted in full, occupying 28,950,528 bytes. The global commitment occupies 48 bytes and the 256 opened seeds occupy 8,192 bytes. With a 32-byte session

identifier, the exact fixed portion of the first message is therefore 28,958,800 bytes (27.62 MiB). The challenge is recomputed and is not transmitted separately. Relative to transmitting all commitment vectors and explicit witnesses, seeded openings reduce the first message by approximately 52.5%.

The aggregate response  $h \in R_q$  occupies

$$|h| = d \lceil \log_2 q \rceil = 64 \cdot 152 = 9,728 \text{ bits} = 1,216 \text{ bytes}.$$

The second message has length  $1,216 + |\pi_{\text{resp}}|$  bytes. An exact total for that message requires an instantiated batched LaBRADOR relation and cannot be inferred from the asymptotic construction.

| Wire component                         | Bytes                         |
|----------------------------------------|-------------------------------|
| 256 unopened commitment vectors        | 28,950,528                    |
| One 48-byte global commitment          | 48                            |
| 256 32-byte opened seeds               | 8,192                         |
| Session identifier                     | 32                            |
| User $\rightarrow$ signer, fixed total | 28,958,800                    |
| Aggregate $h$                          | 1,216                         |
| Signer $\rightarrow$ user              | $1,216 +  \pi_{\text{resp}} $ |
| MAC $(\rho_0, \dots, \rho_{255}, v)$   | 8,208                         |
| MAC plus 256 32-byte messages          | 16,400                        |

Table 1: Preliminary exact wire sizes with seeded openings and one 384-bit global branch commitment under the stated canonical encoding. The batched response proof is not yet instantiated.

The MAC calculation uses 256 tags of 32 bytes and  $d \log_2 p = 128$  bits for  $v$ :

$$|\text{MAC}| = 256 \cdot 32 + 16 = 8,208 \text{ bytes}.$$

If the messages are considered part of the stored credential rather than external signed data, 8,192 additional bytes give a 16,400-byte credential.

## 8.2 Optional optimization: higher-arity cut-and-choose

The binary construction above is the base scheme: it gives the simplest protocol description, two-fork extractor, and security game. Higher arity is an optional optimization and should be analyzed only after the binary scheme is proved secure.

The dominant term can be reduced by replacing binary pairs with groups of  $A = 2^a$  branches. Fiat–Shamir selects one branch per group to remain unopened; the other  $A - 1$  branches are disclosed as 32-byte seeds. If there are  $g$  independent groups, making every selected branch malformed succeeds with probability

$$A^{-g} = 2^{-ag}.$$

For a conservative 128-bit post-quantum grinding target we require  $ag \geq 256$ . Only  $g$  full lattice vectors are then transmitted. Ignoring the response proof, the fixed first-message size is

$$g|\mathbf{c}| + g(A - 1) \cdot 32 + 48 + 32 \text{ bytes}, \quad g = \left\lceil \frac{256}{\log_2 A} \right\rceil.$$

For bandwidth alone, the stated parameters are minimized near  $A = 512$ , where the first message is 3,753,840 bytes (3.58 MiB): 29 full vectors consume 3,279,552 bytes, 14,819 opened seeds consume 474,208 bytes, and the global commitment plus session identifier consume 80 bytes. The corresponding MAC contains 29 tags and one 16-byte rounded value, hence

$$29 \cdot 32 + 16 = 944 \text{ bytes}.$$

This setting requires generating and checking 14,848 branches, however, versus 512 in the binary construction. Values such as  $A = 16$  or  $A = 32$  may therefore give a better end-to-end tradeoff: they reduce issuance to 6.93 or 5.66 MiB while increasing branch computation by only two to three times. A benchmark must include deterministic expansion, hash-to-ring, matrix–vector products, and response-proof scaling before selecting  $A$ .

| $A$  | $g$ | challenge bits | generated branches | first message |
|------|-----|----------------|--------------------|---------------|
| 2    | 256 | 256            | 512                | 27.62 MiB     |
| 4    | 128 | 256            | 512                | 13.82 MiB     |
| 16   | 64  | 256            | 1,024              | 6.93 MiB      |
| 32   | 52  | 260            | 1,664              | 5.66 MiB      |
| 64   | 43  | 258            | 2,752              | 4.72 MiB      |
| 256  | 32  | 256            | 8,192              | 3.70 MiB      |
| 512  | 29  | 261            | 14,848             | 3.58 MiB      |
| 1024 | 26  | 260            | 26,624             | 3.62 MiB      |

Table 2: Bandwidth/computation tradeoff for higher-arity cut-and-choose. The branch count approximates user generation and signer opening work.

The two-fork extraction argument generalizes: two accepting transcripts whose selected indices differ in some group cause the second transcript to open the branch selected in the first. The proof must use an  $A$ -ary challenge and bind all branches in the global commitment.

This optimization changes the key shape and is not merely a wire encoding. In the binary base scheme there are  $\kappa = 256$  independent component secrets

$$(\mathbf{s}_0, \dots, \mathbf{s}_{255}) \in (R_q^n)^{256}.$$

In the  $A$ -ary scheme there are only  $g = \lceil 256 / \log_2 A \rceil$  component secrets,

$$(\mathbf{s}_0, \dots, \mathbf{s}_{g-1}) \in (R_q^n)^g, \quad h = \sum_{i=0}^{g-1} \mathbf{s}_i^T \mathbf{c}_i^* + e.$$

Thus arity changes the public and secret key sizes, the number of terms and noise contributions in the aggregate, the batched response-proof statement, and the number of  $(m_i, \rho_i)$  pairs in the MAC. Each component key retains the full BLNS23 module dimension; only the number of independently keyed groups changes. A concrete  $A$  must therefore receive its own correctness, hardness, and parameter analysis rather than inheriting the binary parameters automatically.

The credential contains  $\kappa$  message/tag pairs unless they admit a safe compression mechanism. This is a major cost and must be included in any comparison. A credible evaluation should report total issuance bytes, prover and verifier time, credential bytes, presentation time, public-key size, secret key size, and rounding-failure probability.

## 9 Conclusion

Aggregate-only cut-and-choose offers a plausible route around the expensive hash-in-ZK issuance proof of BLNS23. Its useful property is all-or-nothing: partial malformed selections remain mixed with at least one honest lane, while removing every honest lane costs  $2^{-\kappa}$ . This intuition is potentially valuable, but it is not yet a theorem. The next step is to formalize and prove the one-honest-lane lemma, or find a related-input counterexample. Only after that step should concrete parameters or performance claims be treated as evidence for a deployable scheme.

## References

- [1] W. Beullens, V. Lyubashevsky, N. Nguyen, and G. Seiler. Lattice-Based Blind Signatures: Short, Efficient, and Round-Optimal. IACR Cryptology ePrint Archive, Report 2023/077, 2023. <https://eprint.iacr.org/2023/077>.
- [2] R. Chairattana-Apirom, L. Hanzlik, J. Loss, A. Lysyanskaya, and B. Wagner. II-Cut-Choo and Friends: Compact Blind Signatures via Parallel Instance Cut-and-Choose and More. IACR Cryptology ePrint Archive, Report 2022/007, 2022. <https://eprint.iacr.org/2022/007>.
- [3] W. Beullens and G. Seiler. LaBRADOR: Compact Proofs for R1CS from Module-SIS. IACR Cryptology ePrint Archive, 2022.



- [4] J. Don, S. Fehr, and C. Majenz. The Measure-and-Reprogram Technique 2.0: Multi-Round Fiat–Shamir and More. IACR Cryptology ePrint Archive, Report 2020/282, 2020. <https://eprint.iacr.org/2020/282>.